

Discussion after meeting

Zhen-Jun Liu

August 27, 2026

Reference: Accelerating multipartite entanglement generation in non-Hermitian superconducting qubits.

Weakly coupled non-Hermitian qubits described by the total non-Hermitian Hamiltonian:

$$\hat{H} = \sum_{j=1}^n \left[\left(\Delta_j - \frac{i\gamma_j}{2} \right) \hat{\sigma}_j^+ \hat{\sigma}_j^- + \Omega_j \hat{\sigma}_j^x \right] + \sum_{j \neq k}^n \sum_{k=1}^n J_{jk} (\hat{\sigma}_j^+ \hat{\sigma}_k^- + \hat{\sigma}_j^- \hat{\sigma}_k^+)$$

Real Part of Eigenvalues (J=0)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.linalg import eig
4 # Define parameters
5 n_qubits = 3 # Number of qubits
6 gamma = 6.0 # Dissipation rate (rad/us)
7 J = 0 # Coupling strength (rad/us)
8 Omega_vals = np.linspace(1.48, 1.52, 1000) # Driving amplitude range
9 # Pauli matrices
10 sigma_x = np.array([[0, 1], [1, 0]])
11 sigma_plus = np.array([[0, 1], [0, 0]])
12 sigma_minus = np.array([[0, 0], [1, 0]])
13 # Single qubit non-Hermitian Hamiltonian
14 H_single = -1j * gamma / 2 * np.array([[1, 0], [0, 0]])
15 # Initialize the total Hamiltonian
16 def build_total_hamiltonian(Omega, J):
17     # Single qubit driving term
18     H_drive = Omega * sigma_x
19     H_single_total = H_single + H_drive
20     # Initialize the total Hamiltonian
21     H_total = np.zeros((2**n_qubits, 2**n_qubits), dtype=complex)
22     # Add the Hamiltonian for each qubit
23     for i in range(n_qubits):
24         identity_list = [np.eye(2)] * n_qubits
```

```

25         identity_list[i] = H_single_total
26         H_total += np.kron(np.kron(
27             identity_list[0], identity_list[1]), identity_list[2]
28         )
29     # Add the coupling terms
30     for i in range(n_qubits):
31         for j in range(i+1, n_qubits):
32             identity_list = [np.eye(2)] * n_qubits
33             identity_list[i] = sigma_plus
34             identity_list[j] = sigma_minus
35             H_total += J * np.kron(
36                 np.kron(identity_list[0], identity_list[1]), identity_list[2]
37             )
38             identity_list[i] = sigma_minus
39             identity_list[j] = sigma_plus
40             H_total += J * np.kron(
41                 np.kron(identity_list[0], identity_list[1]), identity_list[2]
42             )
43     return H_total
44 # Store eigenvalues
45 E_real = [] # Real part
46 E_imag = [] # Imaginary part
47 # Scan Omega
48 for Omega in Omega_vals:
49     H = build_total_hamiltonian(Omega, J)
50     eigenvalues, _ = eig(H)
51     E_real.append(np.real(eigenvalues))
52     E_imag.append(np.imag(eigenvalues))
53 # Convert to numpy arrays
54 E_real = np.array(E_real)
55 E_imag = np.array(E_imag)
56 # Plot Re[E] vs Omega
57 plt.figure(figsize=(6, 4))
58 for i in range(2*n_qubits):
59     plt.scatter(Omega_vals, E_real[:, i], s=5)
60 plt.xlabel("Omega (rad/us)")
61 plt.ylabel("Re[E] (rad/us)")
62 plt.ylim(-0.5, 0.5)
63 plt.xlim(1.48, 1.52)
64 plt.title("Real-Part-of-Eigenvalues-(J=0)")
65 plt.grid()
66 plt.show()

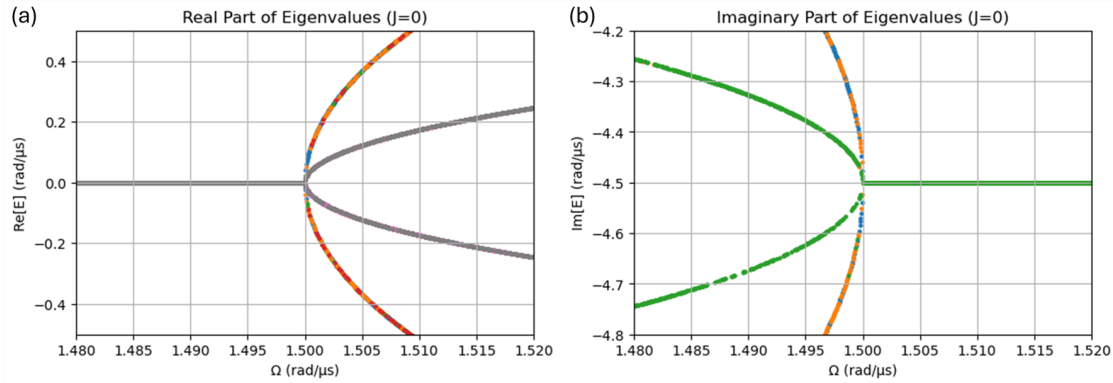
```

Imaginary Part of Eigenvalues (J=0)

```

1 plt.figure(figsize=(6, 4))
2 for i in range(n):
3     plt.scatter(Omega_vals, E_imag[:, i], s=5)
4 plt.xlabel("Omega (rad/us)")
5 plt.ylabel("Im[E] (rad/us)", )
6 plt.ylim(-4.8, -4.2)
7 plt.xlim(1.48, 1.52)
8 plt.title("Imaginary Part of Eigenvalues (J=0)")
9 plt.grid()
10 plt.show()

```



Real Part of Eigenvalues ($J=10^{-3}$)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.linalg import eig
4 # Define parameters
5 n_qubits = 3 # Number of qubits
6 gamma = 6.0 # Dissipation rate (rad/us)
7 J = 1e-3 # Coupling strength (rad/us)
8 Omega_vals = np.linspace(1.48, 1.52, 1000)
9 # Pauli matrices
10 sigma_x = np.array([[0, 1], [1, 0]])
11 sigma_plus = np.array([[0, 1], [0, 0]])
12 sigma_minus = np.array([[0, 0], [1, 0]])
13 # Single qubit non-Hermitian Hamiltonian
14 H_single = -1j * gamma / 2 * np.array([[1, 0], [0, 0]])
15 # Initialize the total Hamiltonian
16 def build_total_hamiltonian(Omega, J):

```

```

17  # Single qubit driving term
18  H_drive = Omega * sigma_x
19  H_single_total = H_single + H_drive
20  # Initialize the total Hamiltonian
21  H_total = np.zeros((2**n_qubits, 2**n_qubits), dtype=complex)
22  # Add the Hamiltonian for each qubit
23  for i in range(n_qubits):
24      identity_list = [np.eye(2)] * n_qubits
25      identity_list[i] = H_single_total
26      H_total += np.kron(np.kron(
27          identity_list[0], identity_list[1]), identity_list[2]
28      )
29  # Add the coupling terms
30  for i in range(n_qubits):
31      for j in range(i+1, n_qubits):
32          identity_list = [np.eye(2)] * n_qubits
33          identity_list[i] = sigma_plus
34          identity_list[j] = sigma_minus
35          H_total += J * np.kron(np.kron(
36              identity_list[0], identity_list[1]), identity_list[2]
37          )
38          identity_list[i] = sigma_minus
39          identity_list[j] = sigma_plus
40          H_total += J * np.kron(np.kron(
41              identity_list[0], identity_list[1]), identity_list[2]
42          )
43  return H_total
44  # Store eigenvalues
45  E_real = [] # Real part
46  E_imag = [] # Imaginary part
47  # Scan Omega
48  for Omega in Omega_vals:
49      H = build_total_hamiltonian(Omega, J)
50      eigenvalues, _ = eig(H)
51      E_real.append(np.real(eigenvalues))
52      E_imag.append(np.imag(eigenvalues))
53  # Convert to numpy arrays
54  E_real = np.array(E_real)
55  E_imag = np.array(E_imag)
56  # Plot Re[E] vs Omega
57  plt.figure(figsize=(6, 4))
58  for i in range(2**n_qubits):
59      plt.scatter(Omega_vals, E_real[:, i], s=5)
60  plt.xlabel("Omega (rad/us)")
61  plt.ylabel("Re[E] (rad/us)")

```

```

62 plt.ylim(-0.51, 0.51)
63 plt.xlim(1.48, 1.52)
64 plt.title("Real Part of Eigenvalues (J=1e-3)")
65 plt.grid()
66 plt.show()

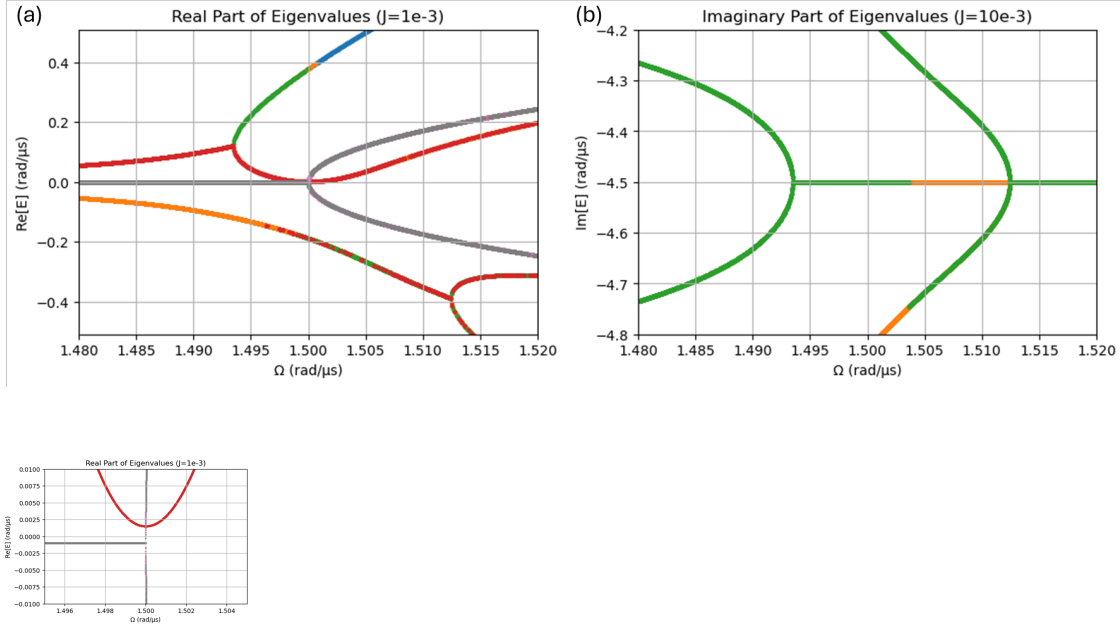
```

Imaginary Part of Eigenvalues ($J=10^{-3}$)

```

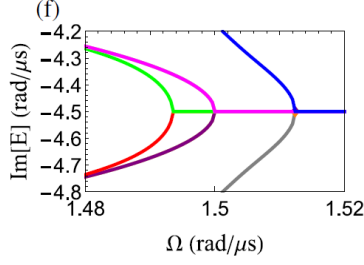
1 plt.figure(figsize=(6, 4))
2 for i in range(n):
3     plt.scatter(Omega_vals, E_imag[:, i], s=5)
4 plt.xlabel("Omega (rad/us)")
5 plt.ylabel("Im[E] (rad/us)", )
6 plt.ylim(-4.8, -4.2)
7 plt.xlim(1.48, 1.52)
8 plt.title("Imaginary Part of Eigenvalues (J=10e-3)")
9 plt.grid()
10 plt.show()

```



There is something wrong with the right hand side picture, in the paper, they simulate exceptional points as shown below:

Tripartite entanglement dynamics of three coupled non-Hermitian qubits.



```

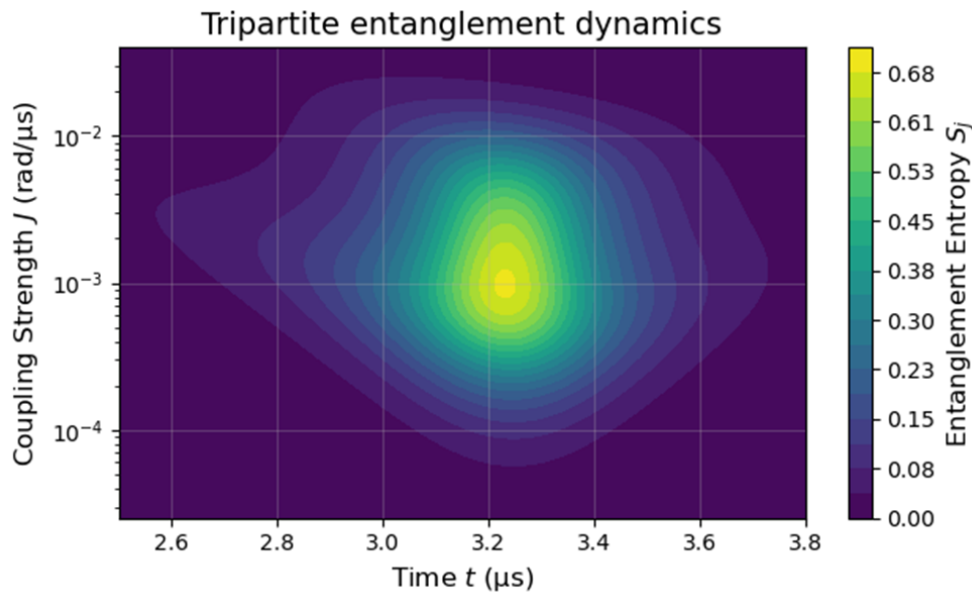
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from qutip import *
4 # ——— System parameters ———
5 n_qubits = 3
6 gamma = 6.0 # Dissipation rate [rad/us]
7 Omega = 1.576 # Driving amplitude [rad/us]
8 J_vals = np.logspace(-4.6, -1.4, 100) # Coupling strength [rad/us]
9 t_vals = np.linspace(2.5, 3.8, 400) # Time [us]
10 f = basis(2, 0)
11 e = basis(2, 1)
12 psi0 = tensor(*[(f - 1j * e).unit() for _ in range(n_qubits)])
13 # ——— Non-Hermitian Hamiltonian ———
14 def build_hamiltonian(J):
15     H = 0
16     for i in range(n_qubits):
17         ops = [qeye(2)] * n_qubits
18         ops[i] = Omega * sigmax() - 1j * (gamma / 2) * projection(2, 1, 1)
19         H += tensor(*ops)
20     # Cyclic nearest-neighbor coupling
21     for i in range(n_qubits):
22         j = (i + 1) % n_qubits
23         for op1, op2 in [(sigmap(), sigmam()), (sigmam(), sigmap())]:
24             ops = [qeye(2)] * n_qubits
25             ops[i] = op1
26             ops[j] = op2
27             H += J * tensor(*ops)
28     return H
29 # ——— Entropy calculation function ———
30 def entanglement_entropy(psi, qubit_idx=0):
31     rho = psi.ptrace(qubit_idx)
32     eigvals = np.clip(np.real(rho.eigenenergies()), 1e-12, 1)
33     return -np.sum(eigvals * np.log(eigvals))
34 # ——— Main calculation loop ———
35 entropy_matrix = np.zeros((len(J_vals), len(t_vals)))
36 for i, J in enumerate(J_vals):

```

```

37     H = build_hamiltonian(J)
38     for j, t in enumerate(t_vals):
39         psi_t = (-1j * H * t).expm() * psi0
40         psi_t = psi_t.unit()
41         entropy_matrix[i, j] = entanglement_entropy(psi_t)
42 # ——— Plotting the contour map (contourf) ———
43 # Rebuild the grid
44 T, JJ = np.meshgrid(t_vals, J_vals)
45 # Plot: Adjust contour levels
46 plt.figure(figsize=(6.5,4))
47 levels = np.linspace(0, 0.72, 20)
48 contour =
49 plt.contourf(T, JJ, entropy_matrix, levels=levels, cmap='viridis')
50 # Logarithmic Y-axis (J axis)
51 plt.yscale('log')
52 # Colorbar with label formatting
53 cbar = plt.colorbar(contour, format='%.2f')
54 cbar.set_label('Entanglement Entropy  $S_j$ ', fontsize=12)
55 # Labels and title
56 plt.xlabel('Time  $t$  (us)', fontsize=12)
57 plt.ylabel('Coupling Strength  $J$  (rad/us)', fontsize=12)
58 plt.title('Tripartite entanglement dynamics', fontsize=14)
59 plt.grid(alpha=0.3)
60 plt.tight_layout()
61 plt.show()

```



```

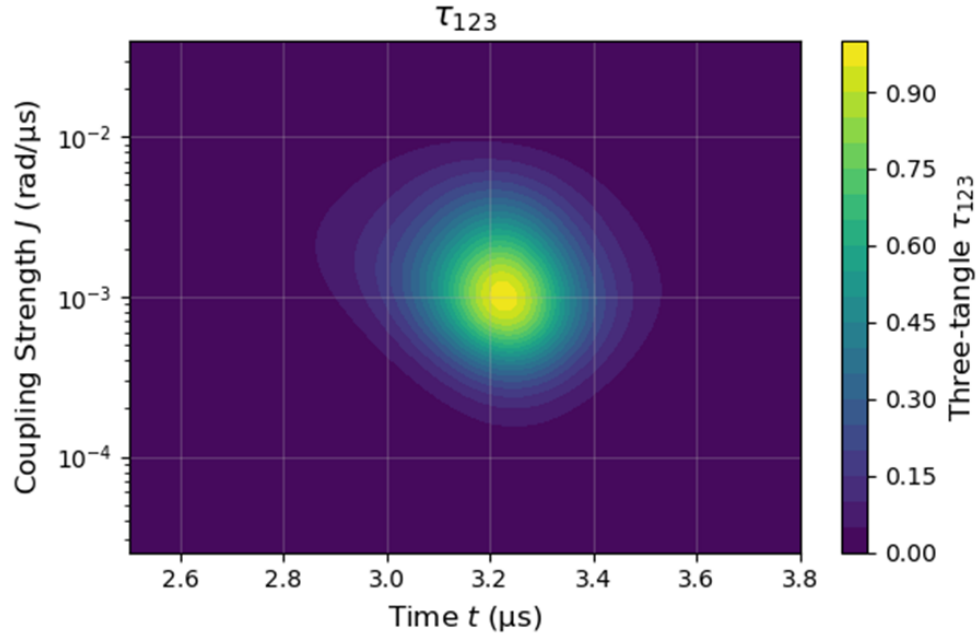
1 import numpy as np

```

```

2 import matplotlib.pyplot as plt
3 from qutip import *
4 n_qubits = 3
5 gamma = 6.0          # Dissipation rate [rad/us]
6 Omega = 1.576        # Driving amplitude [rad/us]
7 J_vals = np.logspace(-4.6, -1.4, 100)    # Coupling strength [rad/us]
8 t_vals = np.linspace(2.5, 3.8, 400)    # Time [us]
9 f = basis(2, 0)
10 e = basis(2, 1)
11 psi0 = tensor(*[(f - 1j * e).unit() for _ in range(n_qubits)])
12 # ——— non Hermitian Hamiltonian ———
13 def build_hamiltonian(J):
14     H = 0
15     for i in range(n_qubits):
16         ops = [qeye(2)] * n_qubits
17         ops[i] = Omega * sigmax() - 1j * (gamma / 2) * projection(2, 1, 1)
18         H += tensor(*ops)
19     # Cyclic nearest-neighbor coupling
20     for i in range(n_qubits):
21         j = (i + 1) % n_qubits
22         for op1, op2 in [(sigmap(), sigmam()), (sigmam(), sigmap())]:
23             ops = [qeye(2)] * n_qubits
24             ops[i] = op1
25             ops[j] = op2
26             H += J * tensor(*ops)
27     return H
28 def concurrence(rho):
29     Y = tensor(sigmay(), sigmay())
30     rho_tilde = Y * rho.conj() * Y
31     R = (rho.sqrtm() * rho_tilde * rho.sqrtm()).eigenenergies()
32     R = np.sort(np.sqrt(np.clip(np.real(R), 0, None)))[::-1]
33     return max(0, R[0] - R[1] - R[2] - R[3])
34 def true_three_tangle(psi):
35     rho1 = psi.ptrace(0)
36     rho12 = psi.ptrace([0, 1])
37     rho13 = psi.ptrace([0, 2])
38     C1_23 = np.sqrt(2 * (1 - (rho1 * rho1).tr()))
39     C12 = concurrence(rho12)
40     C13 = concurrence(rho13)
41     tau = C1_23**2 - C12**2 - C13**2
42     return np.clip(tau, 0, 1)
43 tau_matrix = np.zeros((len(J_vals), len(t_vals)))
44 for i, J in enumerate(J_vals):
45     H = build_hamiltonian(J)
46     for j, t in enumerate(t_vals):

```

```

47     psi_t = (-1j * H * t).expm() * psi0
48     psi_t = psi_t.unit()
49     tau_matrix[i, j] = true_three_tangle(psi_t)
50 T, J = np.meshgrid(t_vals, J_vals)
51
52 plt.figure(figsize=(6, 4))
53 levels = np.linspace(0, 1, 21)
54 contour = plt.contourf(T, J, tau_matrix, levels=levels, cmap='viridis')
55
56 plt.yscale('log')
57 cbar = plt.colorbar(contour)
58 cbar.set_label('Three-tangle- $\tau_{123}$ ', fontsize=12)
59
60 plt.xlabel('Time- $t$ -(us)', fontsize=12)
61 plt.ylabel('Coupling-Strength- $J$ -(rad/us)', fontsize=12)
62 plt.title(' $\tau_{123}$ ', fontsize=14)
63
64 plt.grid(alpha=0.3)
65 plt.tight_layout()
66 plt.show()

```

To be continue...

